The deduction ORDER was decoration.

Training without it costs nothing resolvable.

Retrained with the deductions inside each round PERMUTED, and scored on the same 1500-instance pool: 0.8393 against the sorted-order baseline's 0.8540. Delta -0.0147, 139 discordant instances won and 161 lost, McNemar exact p = 0.2253.

WHY THIS RUN EXISTED. solve_with_dd_trace emits sorted(...) per round, so a round's deductions arrive in ascending node index for 100% of rounds. A hand rule with no deduction in it -- 'the smallest index above the previous one whose colour is still unknown' -- scores 0.632 on next_node, ABOVE the model's own best SAE feature at 0.545 (finding 21). So 'it knows which node comes next' was free, and the open question was whether the model was LEANING on it. Only a retrain can ask: permuting at eval time only puts the model out of distribution.

HOW MUCH COST THIS RULES OUT. The paired 95% interval on the delta is [-0.0373, +0.0080], so this rules out a cost larger than ~3.7 points -- not all cost. The point estimate does favour the sorted arm, and at this n a 1.5-point effect is simply not resolvable.

job 49840619, COMPLETED 05:50:50, ExitCode 0:0 | commit 77ebb35 | scripts/slurm/sudoku2_n24_eventorder.sbatch | seed 0 | shuffle_adj=True, shuffle_events=True

What the model actually sees

Every token is one categorical id into a 34-symbol vocabulary (N=24, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING BEGIN_ROUND SOLVED UNRESOLVED	*section + round markers*
6-9	COLOR_0 COLOR_1 COLOR_2 COLOR_3	*the q colours*
10-33	N0 N1 ... N23	*one id per node, no structure in the id*

GRAMMAR

BEGIN_HINTS	(NODE COLOR)*
BEGIN_GRAPH	(NODE (NODE){degree})*
BEGIN_REASONING	(BEGIN_ROUND (NODE COLOR)+)*
SOLVED	

The graph section is a neighbour table, not an image: node, then its degree neighbours. THE ORDER WITHIN A ROW IS THE LEAK of finding 16.

THIS INSTANCE, 204 TOKENS (seed 50000000038)

HINTS	BEGIN_HINTS N0 COLOR_2 N1 COLOR_1 N4 COLOR_1 N5 COLOR_1 N7 COLOR_2 N9 COLOR_0 ... (23 tokens)
GRAPH	BEGIN_GRAPH N0 N17 N21 N6 N13 N9 N1 N19 N12 N6 N7 N14 ... (145 tokens)
REASONING	BEGIN_REASONING BEGIN_ROUND N3 COLOR_3 N21 COLOR_3 N23 COLOR_3 BEGIN_ROUND N6 COLOR_0 N15 COLOR_1 ... (36 tokens)

Loss is on the REASONING section only: 8 rounds, 13 deductions, 26 target tokens. Everything before BEGIN_REASONING is the problem statement.

One real reasoning trace, as the generator emits it

11 hints of 24 nodes, then 8 rounds of simultaneous deduction. Hints: N0=2 N1=1 N4=1 N5=1 N7=2 N9=0 N10=2 N12=0 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N3 COLOR_3 N21 COLOR_3 N23 COLOR_3
  3 deduced
BEGIN_ROUND          N6 COLOR_0 N15 COLOR_1
  2 deduced
BEGIN_ROUND          N11 COLOR_3 N13 COLOR_0
  2 deduced
BEGIN_ROUND          N18 COLOR_2 N20 COLOR_0
  2 deduced
... 4 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N3, neighbours [17, 4, 10, 19, 5]

colours known before this round:

```
   [0, 1, 2]  (3 = q-1)
```

only colour left: 3

`so the trace emits  N3 COLOR_3`

The model is never shown this reasoning -- only the node/colour pair.

A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

Every round is in ascending node index. That is a convention of solve_with_dd_trace, not a fact about the graph -- and it is what this run removed.

T=8 rounds, the 90th percentile of 40 draws from this config (range 1-10); seed 50000000038, shuffle_adj=True, shuffle_events=False; evidence from sudoku_tokens.evidence_rounds (pre-round state, as DD saw it)

The SAME instance, events permuted within each round

11 hints of 24 nodes, then 8 rounds of simultaneous deduction. Hints: N0=2 N1=1 N4=1 N5=1 N7=2 N9=0 N10=2 N12=0 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND          N3 COLOR_3 N21 COLOR_3 N23 COLOR_3
  3 deduced
BEGIN_ROUND          N15 COLOR_1 N6 COLOR_0
  2 deduced
BEGIN_ROUND          N13 COLOR_0 N11 COLOR_3
  2 deduced
BEGIN_ROUND          N18 COLOR_2 N20 COLOR_0
  2 deduced
... 4 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N3, neighbours [17, 4, 10, 19, 5]

colours known before this round:

```
   [0, 1, 2]  (3 = q-1)
```

only colour left: 3

```
so the trace emits  N3 COLOR_3
```

The model is never shown this reasoning -- only the node/colour pair.

A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

Same graph, same hints, same rounds as a SET, same T, same token count. The ONLY difference from the page before: 4 of these 8 rounds have more than one deduction, and 2 of those came out reordered (rounds 2 and 3 above). A single-deduction round cannot be permuted -- which is why the effect is small to look at and still removes the convention entirely.

T=8 rounds, the 90th percentile of 40 draws from this config (range 1-10); seed 50000000038, shuffle_adj=True, shuffle_events=True; evidence from sudoku_tokens.evidence_rounds (pre-round state, as DD saw it)

The control, and why it is clean

A DD round deduces every node whose neighbours already cover q-1 colours SIMULTANEOUSLY, so the order inside it is an arbitrary presentation choice.

VERIFIED ON CPU BEFORE SUBMITTING (scripts/verify_shuffle_events.py, 300 instances)

- adj, the planted colouring and the hints are identical instance by instance
- every round's membership is identical AS A SET, and T is identical
- the encoded sequence LENGTH is identical, so max_len and n_params cannot move
- 520 of the 707 multi-event rounds (73.6%) actually come out reordered; a 2-event round is left sorted half the time by chance, which is why it is not 100%

	sorted	permuted	change
n_params	9,548,834	9,548,834	identical, as predicted
max_len	210	210	identical, as predicted
final_val_nll	1.3087	1.3410	+0.0323 vs floor +0.0270
final_val_accuracy	0.5086	0.4944	-0.0142 vs floor -0.0163

TOKEN METRICS ARE NOT COMPARABLE ACROSS THIS FLAG -- and that is measurable, not a hedge.

Permuting a round of k simultaneous deductions injects log(k!) nats that NO model can predict. Measured over evaluate()'s own denominator (every valid token, graph section included) that floor is +0.0270 nats/token and -0.0163 token accuracy. The permuted arm lands at +0.0323 and -0.0142: one entropy floor away and no further, so the permutation cost no predictive capacity elsewhere in the sequence. That is why solve_rate, which is order-insensitive, is the headline and val_nll is only a check on the control itself.

floors from scripts/verify_shuffle_events.py; the sorted column is job 49659502 (finding 17), one change away

The pairing is the reason this comparison exists

WHY TWO CELLS TRAINED ON DIFFERENT TARGET ORDERS CAN SHARE A POOL

sudoku_agg_eval prompts with full_tokens[:reasoning_start] -- hints and graph only -- and scores FREE generation against the planted colouring through an order-insensitive check_solution. The one thing this flag changes, the order of events inside a round of the TARGET trace, lands entirely in the discarded suffix. So the two cells are handed byte-identical prompts on the same 1500 instances and McNemar applies. shuffle_events is therefore deliberately NOT in INSTANCE_KEYS, unlike shuffle_adj which changes the prompt itself. (Every teacher-forced analysis -- SAE, probes, circuits, depth slots, attention -- is the other way round, and all nine call sites now pass the flag through.)

adjacency order	solve_rate	delta	exact p
identity	0.8575	reference	--
within_block	0.8550	-0.0025	1.0000
shuffle	0.8775	+0.0200	0.2559
byid	0.8550	-0.0025	1.0000
bycolor	0.8450	-0.0125	0.5224

THE OTHER ORDER AXIS IS STILL DEAD FLAT

The same graphs re-scored under all five adjacency orderings, n=400 each. Nothing resolves, including bycolor -- which RE-INSTATES finding 16's leak and still does not help (-0.0125, p=0.52).

This replicates finding 17b on a model trained without the event order as well, so removing one order convention did not make the model fall back on the other. Both are decoration for this cell.

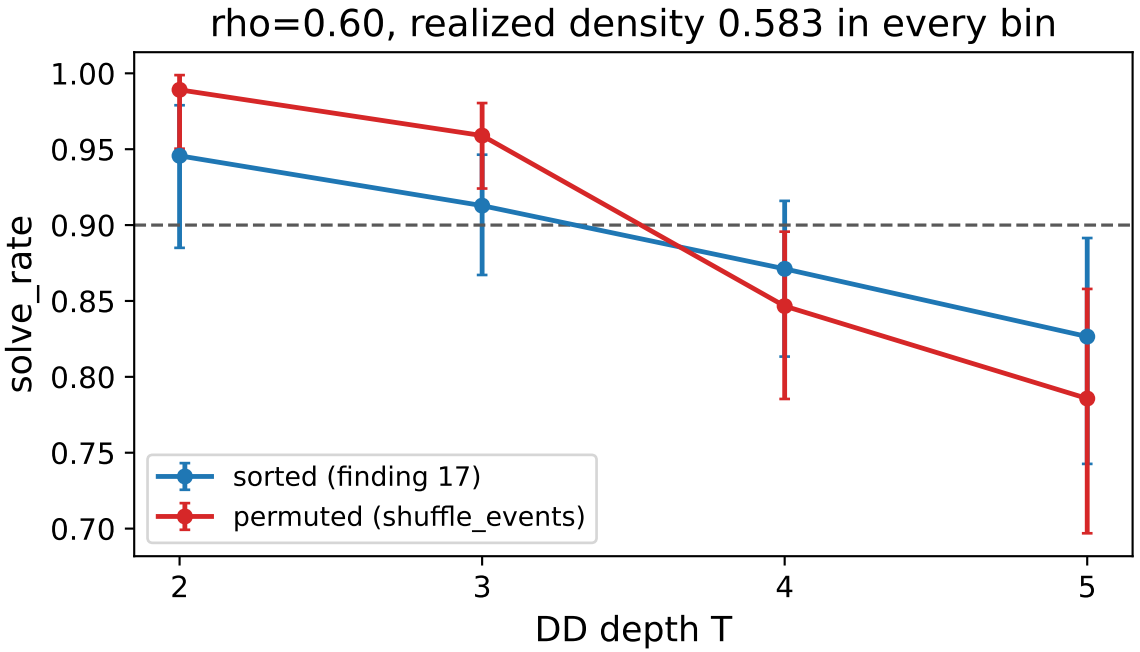
WHAT A NULL HERE IS AND IS NOT

It is: evidence that the ascending-index convention was not the mechanism producing traces, so finding 17's leak-free numbers stand as deduction scores and the open problem that finding 21 raised closes. It is NOT: proof of no effect. 139 won against 161 lost is a 1.5-point point estimate in the sorted arm's favour that this n cannot resolve, and a cost of up to ~3.7 points is inside the interval.

sudoku_agg_eval n=1500 seed_start 5e10; orderings from sudoku_memorization_eval --n-ordering 400

The depth cost survives, and the drop is bigger

Pre-registration (c): r at rho=0.60 stays below -0.8. It does, at -0.979.



	T=2	T=3	T=4	T=5	r	T2-T5
sorted	0.9457	0.9128	0.8712	0.8265	-0.998	0.119
permuted	0.9891	0.9590	0.8466	0.7857	-0.979	0.203

n per bin: T=2:92, T=3:195, T=4:163, T=5:98; identical bins, same generator stream

rho=0.75, THE SHALLOW CONTROL

T=1 1.0000 (n=105), T=2 0.9915 (352), T=3 0.9725 (109), against the sorted arm's 1.000 / 0.969 / 0.890. The permuted model is AHEAD in two of three shallow bins, and well_formed_rate is 1.0 with truncated_rate 0.0 in every bin of both runs -- so nothing here is a grammar failure.

THE SHAPE, NOT JUST THE SLOPE

The permuted arm is BETTER than sorted at T=2 and T=3 and WORSE at T=4 and T=5 -- the two curves cross. So the 1.5-point pool delta is not a uniform tax: it is a redistribution towards the shallow end, and the T=2..5 drop nearly doubles (0.203 vs 0.119). r is nonetheless a hair less negative (-0.979 vs -0.998) because r scores linearity too, and this curve bends -- which is why both columns are shown.

The deepest bin is where the order may still be doing something, and also where n is smallest (98 instances at T=5, interval 0.70-0.86). One targeted follow-up, not a claim.

sudoku_depth_eval --rho 0.75 0.60 --n-pool 600 --min-per-bin 40

Pre-registrations, as resolved

Written into scripts/slurm/sudoku2_n24_eventorder.sbatch before the job ran.

	claim	verdict	evidence
(a)	solve_rate falls RESOLVABLY but stays above 0.75	FALSIFIED (toward 'no change')	0.8393 vs 0.8540, -0.0147, p=0.2253. The named falsifier -- 'the order was pure decoration' -- holds.
(b)	val_nll rises by about the irreducible floor and no more	CONFIRMED	+0.0323 vs floor +0.0270; accuracy -0.0142 vs floor -0.0163; n_params and max_len unmoved
(c)	The depth cost survives: r at rho=0.60 stays below -0.8	CONFIRMED	r = -0.979, and the T=2..5 drop is nearly twice the sorted arm's (0.203 vs 0.119)
(d)	The memorization gap does NOT shrink below +0.1000	NOMINALLY VIOLATED, not resolvably	+0.0817 (train 0.9333 vs held-out 0.8517, p=5.0e-06). Against +0.1000 that is z=-0.76, p=0.45 -- a tie.

(a) IS A FALSIFIED PREDICTION I AM GLAD ABOUT, WHICH IS ITS OWN RISK.

I predicted a resolvable fall, and pre-registered what a null would mean so I could not reinterpret it afterwards: 'the ascending order was pure decoration, the min_unknown shortcut was never being used, and the open problem closes with the leak-free numbers intact.' That is the reading I owe. The honest limit is on the previous page: a null at n=1500 bounds the cost at ~3.7 points, it does not drive it to zero.

(d)'s difference-in-differences uses independent binomial SEs on four rates, which is conservative: both cells trained on the same 80000 instances

What this settles

SETTLED

- The open problem finding 21 raised is CLOSED. The ascending-index order inside a round was available to the model and worth 0.632 to a hand rule, but a model trained without it solves the task just as well, so it was not the mechanism.
- Finding 17's leak-free solve_rates stand as deduction scores, not partly order-decoding scores -- which was the specific worry this run was built to answer.
- Both order conventions in the data are now controlled: shuffle_adj (finding 16) changed the answer dramatically; shuffle_events does not change it at all.

STILL OPEN

- The curves cross at T=4: the permuted arm is ahead in shallow bins and behind in deep ones. If real, the order helps exactly where the reasoning is longest, which is the interesting version of the hypothesis. n=98 in the deepest bin cannot settle it.
- The confound stated up front in the sbatch: the permuted arm's training distribution has strictly higher entropy at the same n_train, so 'lost a crutch' and 'got less usable supervision per instance' are not separated here. Job 49705064 now shows this task IS data-bound at 80000, which makes the confound a live alternative, not a formality.

WHAT I WOULD DO NEXT

1. Re-run this control at the 240000-instance budget that 49705064 established. If both arms then sit at 1.000 the question is closed for good; if the permuted arm alone falls short, the entropy confound was the whole 1.5-point delta.
2. A depth-targeted pool (rho=0.55, n>=400 at T>=5) to test the crossing directly.
3. Re-score the SAE next_node features on THIS checkpoint: a model trained without the order should not have them at all, which is a sharp prediction finding 21 can test.